ByteMart Digital

THE LEETCODE PRACTICE SERIES

Java LeetCode Solutions

76 classic LeetCode problems across 14 core patterns, solved in Java

76 classic LeetCode-style problems across 14 core patterns, each tagged Easy / Medium / Hard, with a worked example, approach, complexity, and Java code.

76	14	3	Full
Problems	Patterns	Levels	Code

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Java LeetCode Solutions

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
This material is provided in good faith for educational and interview-preparation purposes. While every effort has been made to ensure technical accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from its use. All code examples are provided "as is".

Trademarks
Java is a registered trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. All other product names and brands are the property of their respective owners and are used for identification purposes only. This is an independent work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

Arrays & Two Pointers 4

Matrix 13

Strings 17

Hashing 24

Linked Lists 29

Stacks & Queues 36

Recursion & Backtracking 41

Searching & Sorting 45

Trees 50

Graphs 58

Heap & Priority Queue 62

Intervals 64

Bit Manipulation 66

Dynamic Programming 68

This bank contains 76 Java interview questions across 14 sections, including 76 coding challenges with full solutions. Use the bookmarks panel to jump to any section or question.

SECTION 01

Arrays & Two Pointers

Q1 Two Sum — find indices of two numbers that add to a target.

EASY

SOLUTION

Given an integer array and a target value, return the indices of the two numbers that add up to the target. Each input has exactly one solution, and you may not reuse the same element.

```text
Input:  nums = [2, 7, 11, 15], target = 9
Output: [0, 1]        (because nums[0] + nums[1] = 2 + 7 = 9)
```

The brute-force approach checks every pair in O(n^2). Instead, keep a hash map from value to index; for each number, look up whether its complement (target minus the number) has already been seen. That turns the search into a single pass.

```java
static int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int need = target - nums[i];
        if (seen.containsKey(need))
            return new int[]{ seen.get(need), i };
        seen.put(nums[i], i);
    }
    return new int[0];   // no pair found
}
```

Time	Space
O(n)	O(n)

TIP

Store each number only after checking for its complement, so you never pair an element with itself.

Q2 Best Time to Buy and Sell Stock — maximize a single trade profit.

EASY

SOLUTION

Given daily prices, find the maximum profit from buying on one day and selling on a later day. If no profit is possible, return 0.

```text
Input:  prices = [7, 1, 5, 3, 6, 4]
Output: 5             (buy at 1, sell at 6)
```

Track the lowest price seen so far as you scan left to right. At each day, the best profit ending there is the current price minus that running minimum. Keep the maximum of those.

```java
static int maxProfit(int[] prices) {
    int minPrice = Integer.MAX_VALUE, best = 0;
    for (int p : prices) {
        if (p < minPrice) minPrice = p;
        else best = Math.max(best, p - minPrice);
    }
    return best;
}
```

Time	Space
O(n)	O(1)

> **NOTE**
> A single pass suffices — you never need to look back, because the running minimum already captures the best buy day so far.

Q3 Move Zeroes — shift all zeros to the end in place. EASY

SOLUTION

Move all zeros in the array to the end while keeping the relative order of the non-zero elements, modifying the array in place.

```text
Input:  nums = [0, 1, 0, 3, 12]
Output: [1, 3, 12, 0, 0]
```

Use a write pointer. Copy each non-zero value forward to the write position, then fill the remaining slots with zeros. This preserves order in a single pass with no extra array.

```java
static void moveZeroes(int[] nums) {
    int insert = 0;
    for (int n : nums)
        if (n != 0) nums[insert++] = n;
    while (insert < nums.length)
        nums[insert++] = 0;
}
```

Time	Space
O(n)	O(1)

Q4 Container With Most Water — maximize the trapped area. MEDIUM

SOLUTION

Each array value is the height of a vertical line. Pick two lines that, together with the x-axis, hold the most water. The area is the shorter height times the distance between the lines.

```text
Input:  height = [1, 8, 6, 2, 5, 4, 8, 3, 7]
Output: 49
```

Start with the widest pair (the two ends) and move inward. At each step the area is limited by the shorter line, so moving the taller pointer can never help — always move the pointer at the shorter line, hoping for a taller one.

```java
static int maxArea(int[] height) {
    int l = 0, r = height.length - 1, max = 0;
    while (l < r) {
        int area = Math.min(height[l], height[r]) * (r - l);
        max = Math.max(max, area);
        if (height[l] < height[r]) l++;
        else r--;
    }
    return max;
}
```

Time	Space
O(n)	O(1)

KEY IDEA
The two-pointer move is justified because the width only shrinks, so the only way to beat the current area is a taller shorter-line — found by advancing the shorter side.

Q5 Maximum Subarray — Kadane's algorithm.

MEDIUM

SOLUTION

Find the contiguous subarray with the largest sum and return that sum. The array can contain negative numbers.

```text
Input:  nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Output: 6            (the subarray [4, -1, 2, 1])
```

At each index decide: extend the current subarray, or start fresh here. The running sum becomes the larger of the current element alone or the current element plus the previous running sum. Track the best running sum seen.

```java
static int maxSubArray(int[] nums) {
    int cur = nums[0], best = nums[0];
    for (int i = 1; i < nums.length; i++) {
        cur = Math.max(nums[i], cur + nums[i]);
        best = Math.max(best, cur);
    }
    return best;
}
```

Time	Space
O(n)	O(1)

WATCH OUT
Initialise both variables to the first element, not 0 — an all-negative array must return its largest (least negative) element, not 0.

Q6 Merge Intervals — combine all overlapping intervals.

MEDIUM

SOLUTION

Given a list of intervals, merge every set of overlapping intervals and return the non-overlapping result.

```text
Input:  [[1,3], [2,6], [8,10], [15,18]]
Output: [[1,6], [8,10], [15,18]]
```

Sort by start time so overlaps are adjacent. Walk the sorted list, and for each interval either append it (no overlap with the last kept interval) or extend the last interval's end to cover it.

```java
static int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
    List<int[]> out = new ArrayList<>();
    for (int[] iv : intervals) {
        int[] last = out.isEmpty() ? null : out.get(out.size() - 1);
        if (last == null || last[1] < iv[0])
            out.add(iv);
        else
            last[1] = Math.max(last[1], iv[1]);
    }
    return out.toArray(new int[0][]);
}
```

Time	Space
O(n log n)	O(n)

> **NOTE**
> The sort dominates the cost; the merge itself is a single linear pass.

Q7 3Sum — all unique triplets that sum to zero.

MEDIUM

SOLUTION

Return all unique triplets in the array that add up to zero. The solution set must not contain duplicate triplets.

```text
Input:  nums = [-1, 0, 1, 2, -1, -4]
Output: [[-1, -1, 2], [-1, 0, 1]]
```

Sort the array, then fix each element and two-pointer the remainder for a pair summing to its negation. Sorting lets you both move the pointers by comparison and skip duplicates cleanly, avoiding repeated triplets.

```java
static List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> res = new ArrayList<>();
    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue;      // skip dup pivot
        int lo = i + 1, hi = nums.length - 1;
        while (lo < hi) {
            int sum = nums[i] + nums[lo] + nums[hi];
            if (sum == 0) {
                res.add(Arrays.asList(nums[i], nums[lo], nums[hi]));
                while (lo < hi && nums[lo] == nums[lo + 1]) lo++;
                while (lo < hi && nums[hi] == nums[hi - 1]) hi--;
                lo++; hi--;
            } else if (sum < 0) lo++;
            else hi--;
        }
    }
    return res;
}
```

Time	Space
O(n^2)	O(1)

> **NOTE**
> The space excludes the output list; sorting makes duplicate-skipping a simple neighbour comparison.

Q8 Product of Array Except Self — no division.

MEDIUM

SOLUTION

Return an array where each position holds the product of all other elements, without using division and in linear time.

```text
Input:  [1, 2, 3, 4]
Output: [24, 12, 8, 6]
```

Each answer is the product of everything to its left times everything to its right. Compute prefix products in one forward pass into the output, then multiply by a running suffix product in a backward pass — using the output array itself for O(1) extra space.

```java
static int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] out = new int[n];
    out[0] = 1;
    for (int i = 1; i < n; i++)
        out[i] = out[i - 1] * nums[i - 1];   // prefix product
    int right = 1;
    for (int i = n - 1; i >= 0; i--) {
        out[i] *= right;                     // multiply by suffix
        right *= nums[i];
    }
    return out;
}
```

Time	Space
O(n)	O(1)

TIP
Avoiding division also sidesteps the zero-element edge case that a divide-the-total approach must special-case.

Q9 Two Sum II — input array is sorted.

MEDIUM

SOLUTION

The array is sorted in ascending order. Return the 1-indexed positions of the two numbers that add up to the target, using constant extra space.

```text
Input:  numbers = [2, 7, 11, 15], target = 9
Output: [1, 2]
```

Because the array is sorted, use two pointers from the ends. If their sum is too small move the left pointer right (larger value); if too large move the right pointer left. This finds the pair without a hash map.

```java
static int[] twoSumSorted(int[] numbers, int target) {
    int lo = 0, hi = numbers.length - 1;
    while (lo < hi) {
        int sum = numbers[lo] + numbers[hi];
        if (sum == target) return new int[]{ lo + 1, hi + 1 };  // 1-indexed
        else if (sum < target) lo++;
        else hi--;
    }
    return new int[0];
}
```

Time	Space
O(n)	O(1)

Q10 Trapping Rain Water — water held between bars. HARD

SOLUTION

Given an elevation map where each value is a bar's height, compute how much rain water can be trapped between the bars after raining.

```text
Input:  [0,1,0,2,1,0,1,3,2,1,2,1]
Output: 6
```

Water above a bar is bounded by the shorter of the tallest bars to its left and right. Two pointers track running left and right maxima; always advance the side with the smaller max, since that side alone determines the trapped water there.

java

```java
static int trap(int[] height) {
    int l = 0, r = height.length - 1;
    int leftMax = 0, rightMax = 0, water = 0;
    while (l < r) {
        if (height[l] < height[r]) {
            leftMax = Math.max(leftMax, height[l]);
            water += leftMax - height[l];
            l++;
        } else {
            rightMax = Math.max(rightMax, height[r]);
            water += rightMax - height[r];
            r--;
        }
    }
    return water;
}
```

Time	Space
O(n)	O(1)

KEY IDEA

Moving the smaller side is safe because its running max is the true bound there — the other side is guaranteed at least as tall.

SECTION 02
Matrix

Q11 Spiral Matrix — traverse in spiral order.

MEDIUM

SOLUTION

Return all elements of an m-by-n matrix in spiral order, starting from the top-left and moving clockwise.

```text
Input:  [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,2,3,6,9,8,7,4,5]
```

Maintain four boundaries — top, bottom, left, right. Walk the top row, right column, bottom row, and left column in turn, shrinking each boundary after use, until the boundaries cross.

```java
static List<Integer> spiralOrder(int[][] matrix) {
    List<Integer> res = new ArrayList<>();
    if (matrix.length == 0) return res;
    int top = 0, bottom = matrix.length - 1;
    int left = 0, right = matrix[0].length - 1;
    while (top <= bottom && left <= right) {
        for (int j = left; j <= right; j++) res.add(matrix[top][j]);
        top++;
        for (int i = top; i <= bottom; i++) res.add(matrix[i][right]);
        right--;
        if (top <= bottom) {
            for (int j = right; j >= left; j--) res.add(matrix[bottom][j]);
            bottom--;
        }
        if (left <= right) {
            for (int i = bottom; i >= top; i--) res.add(matrix[i][left]);
            left++;
        }
    }
    return res;
}
```

Time	Space
O(m n)	O(1)

NOTE

The inner guards (top <= bottom, left <= right) prevent re-reading a row or column in non-square matrices.

Q12 Rotate Image — rotate a matrix 90 degrees in place.

MEDIUM

SOLUTION

Rotate an n-by-n matrix 90 degrees clockwise, modifying it in place with no extra matrix.

```text
Input:  [[1,2,3],[4,5,6],[7,8,9]]
Output: [[7,4,1],[8,5,2],[9,6,3]]
```

A clockwise rotation equals transposing the matrix (swap across the diagonal) and then reversing each row. Both steps are in place, giving the rotation without extra storage.

```java
static void rotate(int[][] matrix) {
    int n = matrix.length;
    for (int i = 0; i < n; i++)              // transpose
        for (int j = i + 1; j < n; j++) {
            int t = matrix[i][j];
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = t;
        }
    for (int i = 0; i < n; i++)              // reverse each row
        for (int j = 0; j < n / 2; j++) {
            int t = matrix[i][j];
            matrix[i][j] = matrix[i][n - 1 - j];
            matrix[i][n - 1 - j] = t;
        }
}
```

Time	Space
O(n^2)	O(1)

Q13 Set Matrix Zeroes — zero out rows and columns in place.

MEDIUM

SOLUTION

If any element is 0, set its entire row and column to 0, doing so in place with constant extra space.

Use the first row and first column as marker storage: for each zero found in the interior, mark its row and column header. Then apply the zeros from those markers, handling the first row and column separately with two boolean flags.

```java
static void setZeroes(int[][] matrix) {
    int m = matrix.length, n = matrix[0].length;
    boolean firstRow = false, firstCol = false;
    for (int j = 0; j < n; j++) if (matrix[0][j] == 0) firstRow = true;
    for (int i = 0; i < m; i++) if (matrix[i][0] == 0) firstCol = true;
    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            if (matrix[i][j] == 0) { matrix[i][0] = 0; matrix[0][j] = 0; }
    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            if (matrix[i][0] == 0 || matrix[0][j] == 0) matrix[i][j] = 0;
    if (firstRow) for (int j = 0; j < n; j++) matrix[0][j] = 0;
    if (firstCol) for (int i = 0; i < m; i++) matrix[i][0] = 0;
}
```

Time	Space
O(m n)	O(1)

WATCH OUT

Record whether the first row and column originally contained a zero before you overwrite them as markers, or you will corrupt those flags.

Q14 Search a 2D Matrix — binary search a sorted grid. MEDIUM

SOLUTION

Each row is sorted left to right, and each row's first value exceeds the previous row's last. Decide whether a target exists in O(log(m*n)).

```text
Input:  [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 16
Output: true
```

Because the rows join into one fully sorted sequence, treat the grid as a single sorted array of length m*n and binary-search it, converting a flat index back to row and column with divide and modulo.

java

```java
static boolean searchMatrix(int[][] matrix, int target) {
    int m = matrix.length, n = matrix[0].length;
    int lo = 0, hi = m * n - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        int val = matrix[mid / n][mid % n];
        if (val == target) return true;
        else if (val < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return false;
}
```

Time	Space
O(log(m n))	O(1)

SECTION 03
Strings

Q15 Valid Palindrome — ignore case and non-alphanumerics.

EASY

SOLUTION

Determine whether a string reads the same forward and backward, considering only letters and digits and ignoring case.

```text
Input:  "A man, a plan, a canal: Panama"
Output: true
```

Use two pointers from both ends. Skip any character that is not alphanumeric, then compare the two characters case-insensitively. If any pair differs it is not a palindrome.

```java
static boolean isPalindrome(String s) {
    int i = 0, j = s.length() - 1;
    while (i < j) {
        while (i < j && !Character.isLetterOrDigit(s.charAt(i))) i++;
        while (i < j && !Character.isLetterOrDigit(s.charAt(j))) j--;
        if (Character.toLowerCase(s.charAt(i))
                != Character.toLowerCase(s.charAt(j)))
            return false;
        i++; j--;
    }
    return true;
}
```

Time	Space
O(n)	O(1)

Q16 Valid Anagram — check if two strings are rearrangements.

EASY

SOLUTION

Return true if the second string is an anagram of the first — the same characters with the same counts, in any order.

```text
Input:  s = "anagram", t = "nagaram"
Output: true
```

Anagrams have identical character frequencies. For lowercase letters, a fixed 26-slot count array is enough: increment for one string and decrement for the other, then verify every count returned to zero.

```java
static boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] count = new int[26];
    for (int i = 0; i < s.length(); i++) {
        count[s.charAt(i) - 'a']++;
        count[t.charAt(i) - 'a']--;
    }
    for (int c : count)
        if (c != 0) return false;
    return true;
}
```

Time	Space
O(n)	O(1)

TIP

For Unicode input beyond a-z, use a HashMap of character to count instead of a fixed array.

Q17 Longest Substring Without Repeating Characters — sliding window.

MEDIUM

SOLUTION

Find the length of the longest substring that contains no repeated character.

```text
Input:  "abcabcbb"
Output: 3            (the substring "abc")
```

Maintain a window and remember the last index of each character. When you meet a character already inside the window, jump the window start to just past its previous occurrence. The best window length seen is the answer.

```java
static int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> last = new HashMap<>();
    int start = 0, best = 0;
    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        if (last.containsKey(c) && last.get(c) >= start)
            start = last.get(c) + 1;
        last.put(c, i);
        best = Math.max(best, i - start + 1);
    }
    return best;
}
```

Time	Space
O(n)	O(k)

> **NOTE**
> The space O(k) is the size of the character set stored in the map; the window itself is tracked with two indices.

Q18 Group Anagrams — cluster words that are anagrams. MEDIUM

SOLUTION

Group a list of strings so that all anagrams of one another are in the same group.

```text
Input:  ["eat", "tea", "tan", "ate", "nat", "bat"]
Output: [["eat","tea","ate"], ["tan","nat"], ["bat"]]
```

Two words are anagrams exactly when their sorted characters match, so use the sorted string as a map key. Each word is appended to the list under its signature, and the map's values are the groups.

```java
static List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> map = new HashMap<>();
    for (String s : strs) {
        char[] c = s.toCharArray();
        Arrays.sort(c);
        String key = new String(c);
        map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
    }
    return new ArrayList<>(map.values());
}
```

Time	Space
O(n k log k)	O(n k)

NOTE

Here n is the number of words and k the maximum word length; sorting each word costs k log k.

Q19 Longest Palindromic Substring — expand around center. MEDIUM

SOLUTION

Return the longest contiguous substring that is a palindrome.

```text
Input:  "babad"
Output: "bab"   ("aba" is also valid)
```

Every palindrome has a center — a character (odd length) or a gap between two characters (even length). Expand outward from each of the 2n-1 centers while the characters match, tracking the longest span found.

```java
static String longestPalindrome(String s) {
    if (s == null || s.length() < 1) return "";
    int start = 0, end = 0;
    for (int i = 0; i < s.length(); i++) {
        int len1 = expand(s, i, i);       // odd-length center
        int len2 = expand(s, i, i + 1);   // even-length center
        int len = Math.max(len1, len2);
        if (len > end - start + 1) {
            start = i - (len - 1) / 2;
            end = i + len / 2;
        }
    }
    return s.substring(start, end + 1);
}

static int expand(String s, int left, int right) {
    while (left >= 0 && right < s.length()
           && s.charAt(left) == s.charAt(right)) {
        left--; right++;
    }
    return right - left - 1;
}
```

Time	Space
O(n^2)	O(1)

Q20 Longest Common Prefix — shared starting substring. EASY

SOLUTION

Find the longest common prefix shared by all strings in the array. Return an empty string if there is none.

```text
Input:  ["flower", "flow", "flight"]
Output: "fl"
```

Start with the first string as the candidate prefix and shrink it from the right until every other string begins with it. If it shrinks to empty, there is no common prefix.

```java
static String longestCommonPrefix(String[] strs) {
    if (strs.length == 0) return "";
    String prefix = strs[0];
    for (int i = 1; i < strs.length; i++) {
        while (!strs[i].startsWith(prefix)) {
            prefix = prefix.substring(0, prefix.length() - 1);
            if (prefix.isEmpty()) return "";
        }
    }
    return prefix;
}
```

Time	Space
O(S)	O(1)

> **NOTE**
> S is the total number of characters across all strings; each is compared at most once against the shrinking prefix.

Q21 Longest Repeating Character Replacement — sliding window. MEDIUM

SOLUTION

You may replace at most k characters. Find the length of the longest substring containing a single repeated letter achievable after those replacements.

```text
Input:  s = "AABABBA", k = 1
Output: 4
```

Slide a window and track the count of the most frequent character inside it. The window is valid when its size minus that max count (the characters needing replacement) is at most k; otherwise shrink from the left. The largest valid window is the answer.

java

```java
static int characterReplacement(String s, int k) {
    int[] count = new int[26];
    int left = 0, maxCount = 0, best = 0;
    for (int right = 0; right < s.length(); right++) {
        maxCount = Math.max(maxCount, ++count[s.charAt(right) - 'A']);
        while (right - left + 1 - maxCount > k) {
            count[s.charAt(left) - 'A']--;
            left++;
        }
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Time	Space
O(n)	O(1)

SECTION 04
Hashing

Q22 Contains Duplicate — detect any repeated value.

EASY

SOLUTION

Return true if any value appears at least twice in the array, and false if every element is distinct.

```text
Input:  [1, 2, 3, 1]
Output: true
```

Add each element to a hash set. The set's add method returns false when the value is already present, which immediately signals a duplicate without a second pass.

```java
static boolean containsDuplicate(int[] nums) {
    Set<Integer> seen = new HashSet<>();
    for (int n : nums)
        if (!seen.add(n)) return true;
    return false;
}
```

Time	Space
O(n)	O(n)

Q23 Subarray Sum Equals K — count subarrays with a given sum.

MEDIUM

SOLUTION

Count how many contiguous subarrays sum exactly to k. Values may be negative.

```text
Input:  nums = [1, 1, 1], k = 2
Output: 2
```

Use running prefix sums. A subarray ending at the current index sums to k when some earlier prefix equals the current prefix minus k. Keep a map of how many times each prefix sum has occurred, seeding it with 0 seen once.

```java
static int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> prefix = new HashMap<>();
    prefix.put(0, 1);
    int sum = 0, count = 0;
    for (int n : nums) {
        sum += n;
        count += prefix.getOrDefault(sum - k, 0);
        prefix.merge(sum, 1, Integer::sum);
    }
    return count;
}
```

Time	Space
O(n)	O(n)

KEY IDEA

The seed prefix.put(0, 1) accounts for subarrays that start at index 0 and sum to k on their own.

Q24 Top K Frequent Elements — the k most common values.

MEDIUM

SOLUTION

Return the k elements that appear most frequently in the array, in any order.

```text
Input:  nums = [1, 1, 1, 2, 2, 3], k = 2
Output: [1, 2]
```

Count frequencies with a map, then keep a size-k min-heap ordered by frequency. Offer every distinct value and drop the least frequent whenever the heap exceeds k, so the k most frequent remain.

```java
static int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int n : nums) freq.merge(n, 1, Integer::sum);

    PriorityQueue<Integer> heap =
        new PriorityQueue<>((a, b) -> freq.get(a) - freq.get(b));
    for (int key : freq.keySet()) {
        heap.offer(key);
        if (heap.size() > k) heap.poll();
    }
    int[] out = new int[k];
    for (int i = k - 1; i >= 0; i--) out[i] = heap.poll();
    return out;
}
```

Time	Space
O(n log k)	O(n)

> **TIP**
> A bounded size-k heap beats sorting all counts (O(n log n)) when k is much smaller than n.

Q25 Longest Consecutive Sequence — O(n) run length.

MEDIUM

SOLUTION

Given an unsorted array, return the length of the longest run of consecutive integers. It must run in O(n) time (no sorting).

```text
Input:  [100, 4, 200, 1, 3, 2]
Output: 4   (the sequence 1, 2, 3, 4)
```

Put all numbers in a hash set. Only start counting from a number whose predecessor is absent — that is the start of a run — then walk forward while the next value exists. Each number is visited at most twice, giving linear time.

```java
static int longestConsecutive(int[] nums) {
    Set<Integer> set = new HashSet<>();
    for (int n : nums) set.add(n);
    int best = 0;
    for (int n : set) {
        if (!set.contains(n - 1)) {          // start of a run
            int length = 1;
            while (set.contains(n + length)) length++;
            best = Math.max(best, length);
        }
    }
    return best;
}
```

Time	Space
O(n)	O(n)

KEY IDEA

The predecessor check ensures each run is counted once from its start, keeping the total work linear despite the inner loop.

Q26 Valid Sudoku — verify board validity.

MEDIUM

SOLUTION

Determine whether a partially filled 9-by-9 Sudoku board is valid: no row, column, or 3-by-3 sub-box may contain a repeated digit. Empty cells are marked with a dot.

Encode each seen digit with its context — its row, its column, and its 3-by-3 box — as distinct keys in a set. If any insertion reports the key already present, a rule is violated.

```java
static boolean isValidSudoku(char[][] board) {
    Set<String> seen = new HashSet<>();
    for (int i = 0; i < 9; i++)
        for (int j = 0; j < 9; j++) {
            char c = board[i][j];
            if (c == '.') continue;
            if (!seen.add("row" + i + c) ||
                !seen.add("col" + j + c) ||
                !seen.add("box" + (i / 3) + (j / 3) + c))
                return false;
        }
    return true;
}
```

Time	Space
O(1)	O(1)

> **NOTE**
> The board is a fixed 81 cells, so the work is constant; the box index is derived by integer-dividing the coordinates by three.

SECTION 05
Linked Lists

Q27 Reverse a Linked List — iterative pointer reversal.

EASY

SOLUTION

Reverse a singly linked list and return the new head. The node definition used throughout this section is a standard value-and-next node.

```java
class ListNode {
    int val;
    ListNode next;
    ListNode(int v) { val = v; }
}
```

Walk the list once, re-pointing each node's next to its predecessor while carrying a reference to the previous node. When you reach the end, the previous pointer is the new head.

```java
static ListNode reverseList(ListNode head) {
    ListNode prev = null, cur = head;
    while (cur != null) {
        ListNode next = cur.next;   // save the rest
        cur.next = prev;            // reverse the link
        prev = cur;                 // advance prev
        cur = next;                 // advance cur
    }
    return prev;
}
```

Time	Space
O(n)	O(1)

Q28 Linked List Cycle — detect a loop (Floyd's algorithm).

EASY

SOLUTION

Determine whether the linked list contains a cycle — a node whose next pointer eventually revisits an earlier node.

Use two pointers moving at different speeds: a slow pointer one step at a time and a fast pointer two steps. If there is a cycle the fast pointer laps the slow one and they meet; if the fast pointer reaches the end, there is no cycle. This uses no extra memory.

© 2026 ByteMart Digital

```java
static boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

Time	Space
O(n)	O(1)

KEY IDEA

The two-pointer (tortoise and hare) trick detects a cycle in constant space — a hash set of visited nodes would also work but costs O(n) memory.

Q29 Merge Two Sorted Lists — combine into one sorted list.

EASY

SOLUTION

Merge two ascending sorted linked lists into a single ascending sorted list and return its head.

```text
Input:  1 -> 2 -> 4  and  1 -> 3 -> 4
Output: 1 -> 1 -> 2 -> 3 -> 4 -> 4
```

Use a dummy head to simplify edge cases. Repeatedly attach the smaller of the two current nodes to the tail, advancing that list. When one list runs out, attach the remainder of the other.

```java
static ListNode mergeTwoLists(ListNode a, ListNode b) {
    ListNode dummy = new ListNode(0), tail = dummy;
    while (a != null && b != null) {
        if (a.val <= b.val) { tail.next = a; a = a.next; }
        else                { tail.next = b; b = b.next; }
        tail = tail.next;
    }
    tail.next = (a != null) ? a : b;   // attach the leftover
    return dummy.next;
}
```

Time	Space
O(n + m)	O(1)

> **TIP**
> A dummy head node removes the special-case logic for choosing the very first element of the result.

Q30 Remove Nth Node From End — one-pass two-pointer. MEDIUM

SOLUTION

Remove the nth node counting from the end of the list and return the head. Aim to do it in a single pass.

```text
Input:  1 -> 2 -> 3 -> 4 -> 5,  n = 2
Output: 1 -> 2 -> 3 -> 5
```

Advance a fast pointer n steps ahead of a slow pointer, then move both until fast reaches the end. Slow now sits just before the target, so relink around it. A dummy head handles removal of the first node cleanly.

```java
static ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode fast = dummy, slow = dummy;
    for (int i = 0; i < n; i++) fast = fast.next;
    while (fast.next != null) {
        fast = fast.next;
        slow = slow.next;
    }
    slow.next = slow.next.next;   // skip the nth-from-end node
    return dummy.next;
}
```

Time	Space
O(n)	O(1)

Q31 Middle of the Linked List — fast and slow pointers.

EASY

SOLUTION

Return the middle node of the list. If there are two middle nodes (even length), return the second one.

```text
Input:  1 -> 2 -> 3 -> 4 -> 5
Output: node 3
```

Move a slow pointer one step and a fast pointer two steps at a time. When the fast pointer runs off the end, the slow pointer is exactly at the middle — a single pass with no length count needed.

```java
static ListNode middleNode(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```

Time	Space
O(n)	O(1)

Q32 Add Two Numbers — digits stored in reverse.

MEDIUM

SOLUTION

Two numbers are given as linked lists of digits in reverse order. Add them and return the sum as a linked list in the same format.

```text
Input:  (2 -> 4 -> 3) + (5 -> 6 -> 4)   [342 + 465]
Output: 7 -> 0 -> 8                      [807]
```

Walk both lists together, summing corresponding digits plus a carry. Emit each digit modulo ten and carry the tens over. A dummy head simplifies building the result, and the loop continues while either list or the carry remains.

```java
static ListNode addTwoNumbers(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0), cur = dummy;
    int carry = 0;
    while (l1 != null || l2 != null || carry != 0) {
        int sum = carry;
        if (l1 != null) { sum += l1.val; l1 = l1.next; }
        if (l2 != null) { sum += l2.val; l2 = l2.next; }
        carry = sum / 10;
        cur.next = new ListNode(sum % 10);
        cur = cur.next;
    }
    return dummy.next;
}
```

Time	Space
O(max(m,n))	O(1)

Q33 **Palindrome Linked List — check in O(1) space.** EASY

SOLUTION

Determine whether a singly linked list reads the same forwards and backwards, aiming for constant extra space.

Find the middle with fast and slow pointers, reverse the second half in place, then compare it node by node against the first half. This avoids copying values into an array.

```java
static boolean isPalindrome(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {   // find middle
        slow = slow.next;
        fast = fast.next.next;
    }
    ListNode prev = null;                         // reverse second half
    while (slow != null) {
        ListNode next = slow.next;
        slow.next = prev;
        prev = slow;
        slow = next;
    }
    ListNode left = head, right = prev;            // compare halves
    while (right != null) {
        if (left.val != right.val) return false;
        left = left.next;
        right = right.next;
    }
    return true;
}
```

Time	Space
O(n)	O(1)

Q34 LRU Cache — design with O(1) operations.

MEDIUM

SOLUTION

Design a cache with a fixed capacity that supports get and put in O(1), evicting the least-recently-used entry when full.

A LinkedHashMap in access order maintains usage order automatically: overriding removeEldestEntry evicts the oldest entry once the size exceeds capacity. This gives a concise, correct O(1) LRU without hand-writing a doubly linked list.

java

```
class LRUCache {
    private final int capacity;
    private final LinkedHashMap<Integer, Integer> map;

    public LRUCache(int capacity) {
        this.capacity = capacity;
        this.map = new LinkedHashMap<>(capacity, 0.75f, true) {
            protected boolean removeEldestEntry(Map.Entry<Integer, Integer> e) {
                return size() > LRUCache.this.capacity;
            }
        };
    }
    public int get(int key) {
        return map.getOrDefault(key, -1);
    }
    public void put(int key, int value) {
        map.put(key, value);
    }
}
```

Time	Space
O(1) ops	O(capacity)

TIP

In an interview, be ready to also show the manual hash-map-plus-doubly-linked-list version, since some interviewers forbid the built-in shortcut.

SECTION 06

Stacks & Queues

Q35 Valid Parentheses — check bracket matching.

EASY

SOLUTION

Given a string of the brackets ( ) [ ] { }, decide whether every opening bracket is closed by the matching type in the correct order.

```text
Input:  "([{}])"     Output: true
Input:  "([)]"       Output: false
```

Push every opening bracket onto a stack. On a closing bracket, the stack top must be its matching opener — otherwise the string is invalid. At the end the stack must be empty, meaning nothing was left open.

```java
static boolean isValid(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    Map<Character, Character> pairs =
        Map.of(')', '(', ']', '[', '}', '{');
    for (char c : s.toCharArray()) {
        if (pairs.containsValue(c)) {
            stack.push(c);                      // opening bracket
        } else if (pairs.containsKey(c)) {      // closing bracket
            if (stack.isEmpty() || stack.pop() != pairs.get(c))
                return false;
        }
    }
    return stack.isEmpty();
}
```

Time	Space
O(n)	O(n)

TIP

Prefer ArrayDeque over the legacy Stack class — it is faster and not burdened by synchronization.

Q36 Min Stack — stack with constant-time minimum.

MEDIUM

SOLUTION

Design a stack that supports push, pop, top, and retrieving the minimum element, all in constant time.

Store the running minimum alongside each value. When pushing, pair the value with the smaller of itself and the current minimum, so the minimum is always available at the top without scanning.

```java
class MinStack {
    private final Deque<int[]> stack = new ArrayDeque<>(); // {value, min}

    public void push(int x) {
        int min = stack.isEmpty() ? x : Math.min(x, stack.peek()[1]);
        stack.push(new int[]{ x, min });
    }
    public void pop()      { stack.pop(); }
    public int  top()      { return stack.peek()[0]; }
    public int  getMin()   { return stack.peek()[1]; }
}
```

Time	Space
O(1) ops	O(n)

KEY IDEA

Carrying the minimum in each frame keeps getMin at O(1); the alternative of scanning on demand would be O(n).

Q37 Daily Temperatures — monotonic stack.

MEDIUM

SOLUTION

For each day, find how many days you must wait until a warmer temperature. If none, the answer for that day is 0.

```text
Input:  [73, 74, 75, 71, 69, 72, 76, 73]
Output: [1, 1, 4, 2, 1, 1, 0, 0]
```

Keep a stack of indices whose warmer day is still unknown, with temperatures decreasing down the stack. When a warmer day arrives, pop every colder index and record the distance. Each index is pushed and popped once, giving linear time.

© 2026 ByteMart Digital

```java
static int[] dailyTemperatures(int[] temps) {
    int[] answer = new int[temps.length];
    Deque<Integer> stack = new ArrayDeque<>();   // indices, decreasing temp
    for (int i = 0; i < temps.length; i++) {
        while (!stack.isEmpty() && temps[i] > temps[stack.peek()]) {
            int prev = stack.pop();
            answer[prev] = i - prev;
        }
        stack.push(i);
    }
    return answer;
}
```

Time	Space
O(n)	O(n)

Q38 Evaluate Reverse Polish Notation — stack evaluation.

MEDIUM

SOLUTION

Evaluate an arithmetic expression given in Reverse Polish (postfix) notation, where each operator follows its operands.

```text
Input:  ["2", "1", "+", "3", "*"]
Output: 9   ((2 + 1) * 3)
```

Push numbers onto a stack. On an operator, pop the two most recent operands, apply it, and push the result. Order matters for subtraction and division, so pop the right operand first. The final stack value is the answer.

java

```
static int evalRPN(String[] tokens) {
    Deque<Integer> stack = new ArrayDeque<>();
    for (String t : tokens) {
        switch (t) {
            case "+" -> stack.push(stack.pop() + stack.pop());
            case "*" -> stack.push(stack.pop() * stack.pop());
            case "-" -> { int b = stack.pop(), a = stack.pop(); stack.push(a - b); }
            case "/" -> { int b = stack.pop(), a = stack.pop(); stack.push(a / b); }
            default  -> stack.push(Integer.parseInt(t));
        }
    }
    return stack.pop();
}
```

Time	Space
O(n)	O(n)

Q39 Implement Queue using Stacks — amortized O(1).

EASY

SOLUTION

Implement a first-in-first-out queue using only two stacks, supporting push, pop, peek, and empty.

Push always goes to an input stack. When a pop or peek is needed and the output stack is empty, pour the input stack into it, reversing the order so the oldest element surfaces. Each element moves between stacks at most once, giving amortized constant time.

java

```
class MyQueue {
    private final Deque<Integer> in = new ArrayDeque<>();
    private final Deque<Integer> out = new ArrayDeque<>();

    public void push(int x) { in.push(x); }

    public int pop()  { shift(); return out.pop(); }
    public int peek() { shift(); return out.peek(); }
    public boolean empty() { return in.isEmpty() && out.isEmpty(); }

    private void shift() {
        if (out.isEmpty())
            while (!in.isEmpty()) out.push(in.pop());
    }
}
```

Time	Space
O(1) amortized	O(n)

SECTION 07

Recursion & Backtracking

Q40 Subsets — generate the power set.

MEDIUM

SOLUTION

Return all possible subsets (the power set) of a set of distinct integers.

```text
Input:  [1, 2, 3]
Output: [], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]
```

Backtracking builds subsets incrementally. At each position choose to include the current element or not, recording the running subset at every step and undoing the choice after exploring it (the classic add / recurse / remove pattern).

```java
static List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), result);
    return result;
}

static void backtrack(int[] nums, int start, List<Integer> cur,
                      List<List<Integer>> result) {
    result.add(new ArrayList<>(cur));            // record current subset
    for (int i = start; i < nums.length; i++) {
        cur.add(nums[i]);                        // choose
        backtrack(nums, i + 1, cur, result);     // explore
        cur.remove(cur.size() - 1);              // un-choose
    }
}
```

Time	Space
O(n * 2^n)	O(n)

NOTE

There are 2^n subsets and copying each costs up to O(n); the O(n) space is the recursion depth plus the current path.

Q41 Permutations — all orderings of the input.

MEDIUM

SOLUTION

Return every possible ordering (permutation) of a list of distinct integers.

```text
Input:  [1, 2, 3]
Output: [1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]
```

Backtrack while tracking which elements are already used. When the current arrangement reaches full length, record it; otherwise try each unused element in turn, marking and unmarking it around the recursive call.

```java
static List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    helper(nums, new ArrayList<>(), new boolean[nums.length], result);
    return result;
}

static void helper(int[] nums, List<Integer> cur, boolean[] used,
                   List<List<Integer>> result) {
    if (cur.size() == nums.length) {
        result.add(new ArrayList<>(cur));
        return;
    }
    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue;
        used[i] = true;  cur.add(nums[i]);
        helper(nums, cur, used, result);
        used[i] = false; cur.remove(cur.size() - 1);
    }
}
```

Time	Space
O(n * n!)	O(n)

Q42 Generate Parentheses — all valid combinations.

MEDIUM

SOLUTION

Given n pairs of parentheses, generate all well-formed (balanced) combinations.

```text
Input:  n = 3
Output: ((())), (()()), (())(), ()(()), ()()()
```

Build the string character by character, tracking how many open and close brackets are used. You may add an open bracket while some remain, and a close bracket only while it would not exceed the opens — which guarantees validity by construction.

© 2026 ByteMart Digital

```java
static List<String> generateParenthesis(int n) {
    List<String> result = new ArrayList<>();
    gen(new StringBuilder(), 0, 0, n, result);
    return result;
}

static void gen(StringBuilder sb, int open, int close, int n,
                List<String> result) {
    if (sb.length() == 2 * n) { result.add(sb.toString()); return; }
    if (open < n) {
        sb.append('(');
        gen(sb, open + 1, close, n, result);
        sb.deleteCharAt(sb.length() - 1);
    }
    if (close < open) {
        sb.append(')');
        gen(sb, open, close + 1, n, result);
        sb.deleteCharAt(sb.length() - 1);
    }
}
```

Time	Space
O(4^n / sqrt(n))	O(n)

KEY IDEA

Enforcing close < open at each step prunes invalid branches early, so only valid strings are ever built.

Q43 Combination Sum — numbers that add to a target (reuse allowed). MEDIUM

SOLUTION

Given distinct positive candidates and a target, return all unique combinations that sum to the target. Each candidate may be used any number of times.

```text
Input:  candidates = [2, 3, 6, 7], target = 7
Output: [2, 2, 3], [7]
```

Backtrack, subtracting the chosen candidate from the remaining target. Passing the same start index (rather than the next) allows reuse; recursion stops when the remainder hits zero (a hit) or goes negative (a dead end).

```java
static List<List<Integer>> combinationSum(int[] candidates, int target) {
    List<List<Integer>> result = new ArrayList<>();
    combo(candidates, 0, target, new ArrayList<>(), result);
    return result;
}

static void combo(int[] c, int start, int remain, List<Integer> cur,
                  List<List<Integer>> result) {
    if (remain == 0) { result.add(new ArrayList<>(cur)); return; }
    if (remain < 0)  return;
    for (int i = start; i < c.length; i++) {
        cur.add(c[i]);
        combo(c, i, remain - c[i], cur, result);   // i (not i+1) allows reuse
        cur.remove(cur.size() - 1);
    }
}
```

Time	Space
O(2^target)	O(target)

SECTION 08

Searching & Sorting

Q44 Binary Search — find a target in a sorted array.

EASY

SOLUTION

Return the index of a target in a sorted array, or -1 if it is absent.

```text
Input:  nums = [-1, 0, 3, 5, 9, 12], target = 9
Output: 4
```

Repeatedly halve the search range. Compare the middle element to the target and discard the half that cannot contain it. Compute the midpoint as low plus half the gap to avoid integer overflow on large bounds.

```java
static int binarySearch(int[] nums, int target) {
    int lo = 0, hi = nums.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Time	Space
O(log n)	O(1)

WATCH OUT

Use lo + (hi - lo) / 2 rather than (lo + hi) / 2 — the naive sum can overflow int for very large indices.

Q45 Search in Rotated Sorted Array — modified binary search.

MEDIUM

SOLUTION

A sorted array has been rotated at an unknown pivot. Find a target's index in O(log n), or return -1.

```text
Input:  nums = [4, 5, 6, 7, 0, 1, 2], target = 0
Output: 4
```

At each step, one half around the midpoint is still sorted. Determine which half is sorted, check whether the target lies within its ordered range, and recurse into the correct half — preserving the logarithmic time despite the rotation.

```java
static int search(int[] nums, int target) {
    int lo = 0, hi = nums.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] == target) return mid;
        if (nums[lo] <= nums[mid]) {                // left half sorted
            if (nums[lo] <= target && target < nums[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else {                                    // right half sorted
            if (nums[mid] < target && target <= nums[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

Time	Space
O(log n)	O(1)

Q46 Kth Largest Element — using a heap.

MEDIUM

SOLUTION

Return the kth largest element in an unsorted array (the kth in sorted order, not the kth distinct).

```text
Input:  nums = [3, 2, 1, 5, 6, 4], k = 2
Output: 5
```

Maintain a min-heap of size k. Offer every element and, whenever the heap grows past k, remove the smallest. The heap then holds the k largest seen, and its root is the kth largest overall.

```java
static int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> heap = new PriorityQueue<>();  // min-heap
    for (int n : nums) {
        heap.offer(n);
        if (heap.size() > k) heap.poll();
    }
    return heap.peek();
}
```

Time	Space
O(n log k)	O(k)

> **TIP**
> A bounded size-k heap uses far less space than sorting the whole array when k is small.

Q47 Find Minimum in Rotated Sorted Array — binary search.

MEDIUM

SOLUTION

A sorted array was rotated at an unknown pivot. Find its minimum element in O(log n).

```text
Input:  [4, 5, 6, 7, 0, 1, 2]
Output: 0
```

Compare the midpoint to the right end. If the middle is larger, the rotation point (and the minimum) lies to its right; otherwise the minimum is at the middle or to its left. Narrow the range until it collapses to the minimum.

```java
static int findMin(int[] nums) {
    int lo = 0, hi = nums.length - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] > nums[hi]) lo = mid + 1;   // min is to the right
        else hi = mid;                            // min is mid or left
    }
    return nums[lo];
}
```

Time	Space
O(log n)	O(1)

> **NOTE**
> Comparing to the right end (not the left) correctly handles the case where the array is not rotated at all.

Q48 Koko Eating Bananas — binary search on the answer.

MEDIUM

SOLUTION

Koko eats bananas from piles at a chosen speed per hour. Find the minimum integer speed that lets her finish all piles within h hours.

```text
Input:  piles = [3, 6, 7, 11], h = 8
Output: 4
```

The feasibility of a speed is monotonic — if a speed works, any higher speed works too — so binary-search the speed between 1 and the largest pile. For each candidate, compute the hours needed with ceiling division and keep the smallest speed that fits in time.

```java
static int minEatingSpeed(int[] piles, int h) {
    int lo = 1, hi = 0;
    for (int p : piles) hi = Math.max(hi, p);
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (hoursNeeded(piles, mid) <= h) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}

static long hoursNeeded(int[] piles, int speed) {
    long hours = 0;
    for (int p : piles) hours += (p + speed - 1) / speed;   // ceil
    return hours;
}
```

Time	Space
O(n log maxPile)	O(1)

KEY IDEA

'Binary search on the answer' works whenever a candidate answer can be checked for feasibility and feasibility is monotonic.

SECTION 09
Trees

Q49 Maximum Depth of a Binary Tree — recursive height.

EASY

SOLUTION

Return the maximum depth: the number of nodes along the longest path from the root down to a leaf. The node type used in this section is a standard binary tree node.

```java
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int v) { val = v; }
}
```

The depth of a tree is one plus the greater of its two subtrees' depths, with an empty tree having depth zero. This maps directly to a short recursion.

```java
static int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

Time	Space
O(n)	O(h)

NOTE

The space O(h) is the recursion stack depth, where h is the tree height — O(log n) if balanced, O(n) in the worst case.

Q50 Invert a Binary Tree — mirror it.

EASY

SOLUTION

Invert the tree so that the left and right children of every node are swapped, producing its mirror image.

```text
     4                    4
   /   \                /   \
  2     7     ->       7     2
 / \   / \            / \   / \
1   3 6   9          9   6 3   1
```

Recursively invert each subtree, then swap the two children at the current node. The base case of an empty node returns null.

```java
static TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    TreeNode left  = invertTree(root.left);
    TreeNode right = invertTree(root.right);
    root.left  = right;
    root.right = left;
    return root;
}
```

Time	Space
O(n)	O(h)

Q51 Validate a Binary Search Tree — range checking. MEDIUM

SOLUTION

Determine whether a binary tree is a valid BST: every node's value must be greater than all values in its left subtree and less than all in its right subtree.

Checking only a node against its direct children is insufficient — a deep descendant can still violate the order. Instead pass down a valid (low, high) range that tightens as you descend; each node must fall strictly inside its range.

```java
static boolean isValidBST(TreeNode root) {
    return validate(root, null, null);
}

static boolean validate(TreeNode node, Integer low, Integer high) {
    if (node == null) return true;
    if (low  != null && node.val <= low)  return false;
    if (high != null && node.val >= high) return false;
    return validate(node.left,  low, node.val)
        && validate(node.right, node.val, high);
}
```

Time	Space
O(n)	O(h)

WATCH OUT

The common mistake is comparing each node only to its immediate children — the range must be carried down the whole path.

Q52 Binary Tree Level Order Traversal — breadth-first search. MEDIUM

SOLUTION

Return the node values grouped by level, from top to bottom and left to right within each level.

```text
Tree:  3 / (9, 20) / 20 -> (15, 7)
Output: [[3], [9, 20], [15, 7]]
```

Use a queue for breadth-first traversal. At the start of each round, the queue holds exactly one level; process that many nodes, collecting their values and enqueuing their children for the next round.

java

```java
static List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        int size = queue.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);
            if (node.left  != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        result.add(level);
    }
    return result;
}
```

Time	Space
O(n)	O(n)

KEY IDEA

Snapshotting the queue size at the start of each loop is what separates one level from the next.

Q53 Diameter of a Binary Tree — longest path between nodes.

EASY

SOLUTION

Return the length (in edges) of the longest path between any two nodes in the tree. The path may or may not pass through the root.

The longest path through any node is the sum of the heights of its left and right subtrees. Compute depths recursively, and while doing so update a running maximum of left-plus-right at every node. A one-element holder carries the best value out of the recursion.

```java
static int diameterOfBinaryTree(TreeNode root) {
    int[] best = new int[1];
    depth(root, best);
    return best[0];
}

static int depth(TreeNode node, int[] best) {
    if (node == null) return 0;
    int left  = depth(node.left, best);
    int right = depth(node.right, best);
    best[0] = Math.max(best[0], left + right);   // path through this node
    return 1 + Math.max(left, right);
}
```

Time	Space
O(n)	O(h)

KEY IDEA
Combining the diameter update with the depth computation keeps it a single O(n) traversal rather than recomputing heights per node.

Q54 Same Tree — check two trees are identical.

EASY

SOLUTION

Determine whether two binary trees are structurally identical and have the same values at every node.

Compare the roots, then recurse on left and right subtrees. Two trees match if both nodes are null, or both are non-null with equal values and matching subtrees; any mismatch short-circuits to false.

```java
static boolean isSameTree(TreeNode p, TreeNode q) {
    if (p == null && q == null) return true;
    if (p == null || q == null || p.val != q.val) return false;
    return isSameTree(p.left, q.left)
        && isSameTree(p.right, q.right);
}
```

Time	Space
O(n)	O(h)

Q55 Lowest Common Ancestor of a BST — use the ordering.

MEDIUM

SOLUTION

Given a binary search tree and two nodes, find their lowest common ancestor — the deepest node that has both as descendants.

Exploit the BST ordering: if both targets are smaller than the current node, go left; if both are larger, go right. The first node where the targets diverge (one on each side, or one equals the node) is their lowest common ancestor.

```java
static TreeNode lowestCommonAncestor(TreeNode root,
                                     TreeNode p, TreeNode q) {
    while (root != null) {
        if (p.val < root.val && q.val < root.val) root = root.left;
        else if (p.val > root.val && q.val > root.val) root = root.right;
        else return root;   // split point = LCA
    }
    return null;
}
```

Time	Space
O(h)	O(1)

Q56 Balanced Binary Tree — height-balanced check.

EASY

SOLUTION

Determine whether a binary tree is height-balanced: for every node the heights of its two subtrees differ by at most one.

Compute heights bottom-up and use a sentinel of -1 to signal imbalance. As soon as any subtree is unbalanced, propagate -1 upward so the whole traversal stops doing real work — giving a single O(n) pass instead of recomputing heights per node.

© 2026 ByteMart Digital

```java
static boolean isBalanced(TreeNode root) {
    return height(root) != -1;
}

static int height(TreeNode node) {
    if (node == null) return 0;
    int left = height(node.left);
    if (left == -1) return -1;
    int right = height(node.right);
    if (right == -1) return -1;
    if (Math.abs(left - right) > 1) return -1;
    return 1 + Math.max(left, right);
}
```

Time	Space
O(n)	O(h)

Q57 Binary Tree Right Side View — last node per level.

MEDIUM

SOLUTION

Return the values visible from the right side of the tree, top to bottom — that is, the rightmost node at each level.

```text
Tree levels: [1], [2, 3], [5, 4]
Output: [1, 3, 4]
```

Run a level-order (breadth-first) traversal and record only the last node processed in each level, since that is the rightmost one. Enqueue children left-to-right so the last dequeued per level is the right edge.

java

```java
static List<Integer> rightSideView(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            if (i == size - 1) res.add(node.val);   // rightmost in level
            if (node.left  != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
    }
    return res;
}
```

Time	Space
O(n)	O(n)

SECTION 10
Graphs

Q58 Number of Islands — count connected land regions.

MEDIUM

SOLUTION

A grid of '1' (land) and '0' (water) is given. Count the islands, where an island is land connected horizontally or vertically.

```text
Input:  [[1,1,0,0],[1,0,0,1],[0,0,1,1]]
Output: 2
```

Scan the grid; each time you meet unvisited land, increment the count and flood-fill (depth-first) all connected land, sinking it to water so it is not counted again. Every cell is visited a constant number of times.

```java
static int numIslands(char[][] grid) {
    int count = 0;
    for (int i = 0; i < grid.length; i++)
        for (int j = 0; j < grid[0].length; j++)
            if (grid[i][j] == '1') {
                sink(grid, i, j);
                count++;
            }
    return count;
}

static void sink(char[][] grid, int i, int j) {
    if (i < 0 || i >= grid.length || j < 0 || j >= grid[0].length
            || grid[i][j] != '1') return;
    grid[i][j] = '0';                // mark visited
    sink(grid, i + 1, j);
    sink(grid, i - 1, j);
    sink(grid, i, j + 1);
    sink(grid, i, j - 1);
}
```

Time	Space
O(m n)	O(m n)

NOTE

The space is the worst-case recursion depth when the whole grid is land; a BFS with an explicit queue avoids deep recursion.

Q59 Clone Graph — deep-copy an undirected graph.

MEDIUM

SOLUTION

Given a reference to a node in a connected undirected graph, return a deep copy of the entire graph. Each node holds a value and a list of neighbours.

Traverse depth-first while keeping a map from original nodes to their clones. Before recursing into a neighbour, check the map to reuse an existing clone — this both avoids infinite loops on cycles and wires up shared neighbours correctly.

```java
// class Node { int val; List<Node> neighbors = new ArrayList<>(); }
static Node cloneGraph(Node node) {
    if (node == null) return null;
    return dfs(node, new HashMap<>());
}

static Node dfs(Node node, Map<Node, Node> clones) {
    if (clones.containsKey(node)) return clones.get(node);
    Node copy = new Node(node.val);
    clones.put(node, copy);
    for (Node nb : node.neighbors)
        copy.neighbors.add(dfs(nb, clones));
    return copy;
}
```

Time	Space
O(V + E)	O(V)

KEY IDEA

Recording a clone in the map before recursing into neighbours is what prevents an infinite loop on a cycle.

Q60 Course Schedule — detect a prerequisite cycle.

MEDIUM

SOLUTION

Given a number of courses and prerequisite pairs, decide whether all courses can be finished — that is, whether the prerequisite graph has no cycle.

```text
Input:  numCourses = 2, prerequisites = [[1, 0]]
Output: true   (take 0, then 1)
```

Use Kahn's topological sort. Build the graph and each node's in-degree, then repeatedly remove nodes with no remaining prerequisites, decrementing their dependents. If every course is removed, there is no cycle; otherwise a cycle blocked some of them.

```java
static boolean canFinish(int numCourses, int[][] prerequisites) {
    List<List<Integer>> graph = new ArrayList<>();
    int[] indegree = new int[numCourses];
    for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());
    for (int[] p : prerequisites) {
        graph.get(p[1]).add(p[0]);
        indegree[p[0]]++;
    }
    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < numCourses; i++)
        if (indegree[i] == 0) queue.offer(i);
    int done = 0;
    while (!queue.isEmpty()) {
        int c = queue.poll();
        done++;
        for (int next : graph.get(c))
            if (--indegree[next] == 0) queue.offer(next);
    }
    return done == numCourses;
}
```

Time	Space
O(V + E)	O(V + E)

Q61 Rotting Oranges — multi-source BFS timing.

MEDIUM

SOLUTION

In a grid, 2 is a rotten orange, 1 is fresh, and 0 is empty. Each minute, rot spreads to adjacent fresh oranges. Return the minutes until none are fresh, or -1 if some can never rot.

This is a breadth-first search from all rotten oranges at once. Seed the queue with every rotten cell, then process the grid level by level — each level is one minute — rotting fresh neighbours. If fresh oranges remain after the BFS, they were unreachable.

```java
static int orangesRotting(int[][] grid) {
    int m = grid.length, n = grid[0].length, fresh = 0;
    Queue<int[]> queue = new LinkedList<>();
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++) {
            if (grid[i][j] == 2) queue.offer(new int[]{ i, j });
            else if (grid[i][j] == 1) fresh++;
        }
    if (fresh == 0) return 0;
    int minutes = 0;
    int[][] dirs = { {1,0}, {-1,0}, {0,1}, {0,-1} };
    while (!queue.isEmpty() && fresh > 0) {
        minutes++;
        for (int k = queue.size(); k > 0; k--) {
            int[] cell = queue.poll();
            for (int[] d : dirs) {
                int r = cell[0] + d[0], c = cell[1] + d[1];
                if (r >= 0 && r < m && c >= 0 && c < n && grid[r][c] == 1) {
                    grid[r][c] = 2;
                    fresh--;
                    queue.offer(new int[]{ r, c });
                }
            }
        }
    }
    return fresh == 0 ? minutes : -1;
}
```

Time	Space
O(m n)	O(m n)

SECTION 11

Heap & Priority Queue

Q62 Merge k Sorted Lists — combine with a heap.

HARD

SOLUTION

Merge k sorted linked lists into one sorted list and return its head.

```text
Input:  [1->4->5, 1->3->4, 2->6]
Output: 1->1->2->3->4->4->5->6
```

Keep a min-heap of the current front node of each list. Repeatedly pop the smallest, append it to the result, and push its successor. The heap always holds at most k nodes, so each of the N nodes costs a log-k heap operation.

```java
static ListNode mergeKLists(ListNode[] lists) {
    PriorityQueue<ListNode> pq =
        new PriorityQueue<>((a, b) -> a.val - b.val);
    for (ListNode node : lists)
        if (node != null) pq.offer(node);
    ListNode dummy = new ListNode(0), tail = dummy;
    while (!pq.isEmpty()) {
        ListNode node = pq.poll();
        tail.next = node;
        tail = node;
        if (node.next != null) pq.offer(node.next);
    }
    return dummy.next;
}
```

Time	Space
O(N log k)	O(k)

NOTE

N is the total number of nodes across all lists; the heap size never exceeds k, the number of lists.

Q63 Find Median from Data Stream — two heaps.

HARD

SOLUTION

Design a structure that ingests a stream of numbers and can return the running median at any time.

Keep a max-heap of the smaller half and a min-heap of the larger half, balanced so their sizes differ by at most one. The median is then either the top of the larger heap or the average of the two tops — all in constant time, with logarithmic inserts.

java

```
class MedianFinder {
    private final PriorityQueue<Integer> low =
        new PriorityQueue<>(Collections.reverseOrder());  // max-heap
    private final PriorityQueue<Integer> high =
        new PriorityQueue<>();                            // min-heap

    public void addNum(int num) {
        low.offer(num);
        high.offer(low.poll());          // balance values across heaps
        if (high.size() > low.size())
            low.offer(high.poll());      // keep low at least as large
    }
    public double findMedian() {
        if (low.size() > high.size()) return low.peek();
        return (low.peek() + high.peek()) / 2.0;
    }
}
```

Time	Space
add O(log n), median O(1)	O(n)

KEY IDEA

Passing each new number through both heaps guarantees the smaller half and larger half stay correctly partitioned.

SECTION 12
Intervals

Q64 Insert Interval — add and merge into a sorted list.

MEDIUM

SOLUTION

Given a list of non-overlapping intervals sorted by start and a new interval, insert it and merge any resulting overlaps, returning the sorted result.

```text
Input:  [[1,3],[6,9]], newInterval = [2,5]
Output: [[1,5],[6,9]]
```

Copy intervals that end before the new one begins, then merge every interval that overlaps the new one by widening its bounds, add the merged interval, and finally copy the rest. Because the input is sorted, this is a single linear pass.

```java
static int[][] insert(int[][] intervals, int[] newInterval) {
    List<int[]> res = new ArrayList<>();
    int i = 0, n = intervals.length;
    while (i < n && intervals[i][1] < newInterval[0])
        res.add(intervals[i++]);                        // before overlap
    while (i < n && intervals[i][0] <= newInterval[1]) {   // merge overlaps
        newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
        newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
        i++;
    }
    res.add(newInterval);
    while (i < n) res.add(intervals[i++]);              // after overlap
    return res.toArray(new int[0][]);
}
```

Time	Space
O(n)	O(n)

Q65 Non-overlapping Intervals — fewest removals (greedy).

MEDIUM

SOLUTION

Return the minimum number of intervals to remove so that the rest do not overlap.

```text
Input:  [[1,2],[2,3],[3,4],[1,3]]
Output: 1   (remove [1,3])
```

Sort by end time and greedily keep intervals that start at or after the last kept end. Keeping the interval that finishes earliest leaves the most room for the rest, so every interval that overlaps the last kept one must be removed.

```java
static int eraseOverlapIntervals(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[1], b[1]));
    int end = Integer.MIN_VALUE, removed = 0;
    for (int[] iv : intervals) {
        if (iv[0] >= end) end = iv[1];   // keep it
        else removed++;                  // overlaps -> remove
    }
    return removed;
}
```

Time	Space
O(n log n)	O(1)

KEY IDEA

Sorting by end time is the crux — greedily finishing earliest maximises how many intervals can remain.

SECTION 13

Bit Manipulation

Q66 Single Number — find the element that appears once.

EASY

SOLUTION

Every element appears exactly twice except one, which appears once. Find that single element in linear time and constant space.

```text
Input:  [4, 1, 2, 1, 2]
Output: 4
```

XOR is the key: a value XORed with itself is zero, and XOR is commutative. XORing all elements cancels every pair and leaves only the unique number — no extra memory required.

```java
static int singleNumber(int[] nums) {
    int result = 0;
    for (int n : nums) result ^= n;   // pairs cancel to 0
    return result;
}
```

Time	Space
O(n)	O(1)

KEY IDEA

XOR gives a constant-space solution where a hash-set count would cost O(n) memory.

Q67 Number of 1 Bits — count set bits (Hamming weight).

EASY

SOLUTION

Count the number of 1 bits in the binary representation of an integer.

```text
Input:  11  (binary 1011)
Output: 3
```

The trick n AND (n - 1) clears the lowest set bit. Repeating it until the number becomes zero runs exactly once per set bit, so the loop count is the answer — faster than checking all 32 bit positions.

java

```java
static int hammingWeight(int n) {
    int count = 0;
    while (n != 0) {
        n &= (n - 1);   // drop the lowest set bit
        count++;
    }
    return count;
}
```

Time	Space
O(bits set)	O(1)

SECTION 14

Dynamic Programming

Q68 Climbing Stairs — count distinct ways.

EASY

SOLUTION

You can climb 1 or 2 steps at a time. Count the distinct ways to reach the top of a staircase with n steps.

```text
Input:  n = 4
Output: 5
```

The ways to reach step n equal the ways to reach step n-1 plus those to reach step n-2 — the Fibonacci recurrence. Compute it bottom-up keeping only the last two values, avoiding exponential recursion.

```java
static int climbStairs(int n) {
    if (n <= 2) return n;
    int a = 1, b = 2;
    for (int i = 3; i <= n; i++) {
        int c = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

Time	Space
O(n)	O(1)

Q69 House Robber — max sum without adjacent picks.

MEDIUM

SOLUTION

Given the amounts in a row of houses, find the maximum you can rob without taking from two adjacent houses.

```text
Input:  [2, 7, 9, 3, 1]
Output: 12            (rob houses 2 + 9 + 1)
```

At each house choose the better of skipping it (keep the previous best) or robbing it (its value plus the best from two houses back). Track the last two results, so only constant memory is needed.

```java
static int rob(int[] nums) {
    int prev = 0, curr = 0;
    for (int n : nums) {
        int take = prev + n;
        prev = curr;
        curr = Math.max(curr, take);
    }
    return curr;
}
```

Time	Space
O(n)	O(1)

Q70 Coin Change — fewest coins for an amount. MEDIUM

SOLUTION

Given coin denominations and a target amount, return the minimum number of coins needed, or -1 if the amount cannot be formed. You have an unlimited supply of each coin.

```text
Input:  coins = [1, 2, 5], amount = 11
Output: 3             (5 + 5 + 1)
```

Build up the best answer for every amount from 1 to the target. For each amount, try every coin that fits and take one plus the best solution for the remainder. Seed the table with an unreachable large value and 0 coins for amount 0.

```java
static int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);   // sentinel = unreachable
    dp[0] = 0;
    for (int i = 1; i <= amount; i++)
        for (int coin : coins)
            if (coin <= i)
                dp[i] = Math.min(dp[i], dp[i - coin] + 1);
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Time	Space
O(amount * coins)	O(amount)

> **NOTE**
> This is a bottom-up DP; a greedy 'largest coin first' strategy does not work for arbitrary denominations.

Q71 Longest Common Subsequence — 2D dynamic programming. MEDIUM

SOLUTION

Find the length of the longest subsequence common to two strings. A subsequence keeps relative order but need not be contiguous.

```text
Input:  a = "abcde", b = "ace"
Output: 3             (the subsequence "ace")
```

Fill a table where each cell is the LCS length of the two prefixes. When the current characters match, extend the diagonal by one; otherwise carry the better of dropping one character from either string. The bottom-right cell holds the answer.

```java
static int longestCommonSubsequence(String a, String b) {
    int m = a.length(), n = b.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++)
            if (a.charAt(i - 1) == b.charAt(j - 1))
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
    return dp[m][n];
}
```

Time	Space
O(m * n)	O(m * n)

> **TIP**
> The space can be reduced to O(n) by keeping only the previous row, since each cell depends only on the row above and the current row.

Q72 Word Break — can the string be segmented?

MEDIUM

SOLUTION

Given a string and a dictionary of words, decide whether the string can be split into a sequence of one or more dictionary words.

```text
Input:  s = "applepenapple", dict = ["apple", "pen"]
Output: true
```

Let dp[i] mean the first i characters can be segmented. It is true when some earlier position j is segmentable and the piece from j to i is a dictionary word. Build the table forward; the final cell answers the question.

```java
static boolean wordBreak(String s, List<String> wordDict) {
    Set<String> words = new HashSet<>(wordDict);
    boolean[] dp = new boolean[s.length() + 1];
    dp[0] = true;
    for (int i = 1; i <= s.length(); i++)
        for (int j = 0; j < i; j++)
            if (dp[j] && words.contains(s.substring(j, i))) {
                dp[i] = true;
                break;
            }
    return dp[s.length()];
}
```

Time	Space
O(n^2)	O(n)

> **NOTE**
> Substring extraction adds a factor of the word length; a trie can reduce repeated prefix checks for large dictionaries.

Q73 Unique Paths — count grid paths.

MEDIUM

SOLUTION

A robot at the top-left of an m-by-n grid can move only right or down. Count the distinct paths to the bottom-right corner.

```text
Input:  m = 3, n = 7
Output: 28
```

The paths to a cell equal the paths from directly above plus those from the left. Roll the 2D table into a single row updated left to right, since each cell needs only the value above (current row) and to the left.

```java
static int uniquePaths(int m, int n) {
    int[] dp = new int[n];
    Arrays.fill(dp, 1);
    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            dp[j] += dp[j - 1];   // above + left
    return dp[n - 1];
}
```

Time	Space
O(m n)	O(n)

Q74 Longest Increasing Subsequence — O(n log n).

MEDIUM

SOLUTION

Find the length of the longest strictly increasing subsequence of an array.

```text
Input:  [10, 9, 2, 5, 3, 7, 101, 18]
Output: 4   (e.g. 2, 3, 7, 101)
```

Maintain a list of the smallest possible tail for each subsequence length. For each number, binary-search the first tail that is not smaller and replace it (or extend the list). The list's length is the answer, though its contents are not an actual subsequence.

```java
static int lengthOfLIS(int[] nums) {
    int[] tails = new int[nums.length];
    int size = 0;
    for (int x : nums) {
        int lo = 0, hi = size;
        while (lo < hi) {
            int mid = (lo + hi) / 2;
            if (tails[mid] < x) lo = mid + 1;
            else hi = mid;
        }
        tails[lo] = x;
        if (lo == size) size++;
    }
    return size;
}
```

Time	Space
O(n log n)	O(n)

> **NOTE**
> A simpler O(n^2) DP also exists; the binary-search version is the optimal one to know for interviews.

Q75 Maximum Product Subarray — track min and max.

MEDIUM

SOLUTION

Find the contiguous subarray with the largest product and return that product. The array may contain negatives and zeros.

```text
Input:  [2, 3, -2, 4]
Output: 6   (the subarray [2, 3])
```

Unlike the sum version, a negative number can flip the largest product to the smallest and vice versa, so track both the running max and min ending at each index. On a negative value, swap them before updating; the overall answer is the best max seen.

```java
static int maxProduct(int[] nums) {
    int max = nums[0], min = nums[0], result = nums[0];
    for (int i = 1; i < nums.length; i++) {
        int n = nums[i];
        if (n < 0) { int t = max; max = min; min = t; }  // swap on negative
        max = Math.max(n, max * n);
        min = Math.min(n, min * n);
        result = Math.max(result, max);
    }
    return result;
}
```

Time	Space
O(n)	O(1)

KEY IDEA

Carrying the running minimum is what handles negatives, since two negatives can multiply into the largest product.

Q76 Jump Game — can you reach the last index? (greedy)

MEDIUM

SOLUTION

Each value is the maximum jump length from that position. Starting at index 0, decide whether you can reach the last index.

```text
Input:  [2, 3, 1, 1, 4]  -> true
Input:  [3, 2, 1, 0, 4]  -> false
```

Track the farthest index reachable so far. Scan left to right; if you ever stand beyond that reach, the end is unreachable. Otherwise extend the reach by the current jump, and success is guaranteed once the reach covers the last index.

java

```java
static boolean canJump(int[] nums) {
    int reach = 0;
    for (int i = 0; i < nums.length; i++) {
        if (i > reach) return false;         // stuck before this index
        reach = Math.max(reach, i + nums[i]);
    }
    return true;
}
```

Time	Space
O(n)	O(1)